

Adam SLAM - the last mile of camera calibration with 3DGS

Matthieu GENDRIN^{1,2}, Stéphane PATEUX¹, Xiaoran JIANG², Théo LADUNE¹, and Luce MORIN²

¹Orange Innovation, France, `firstname.lastname@orange.com`

²Univ Rennes, INSA Rennes, CNRS, IETR (UMR 6164), France, `firstname.lastname@insa-rennes.fr`

Résumé

La qualité de la calibration des caméras est d'une importance majeure pour évaluer les progrès dans la synthèse de nouvelles vues, une erreur de 1 pixel sur la calibration ayant un impact significatif sur la qualité de reconstruction. Etant donné qu'il n'y a pas de vérité terrain pour les scènes réelles, la qualité de la calibration est évaluée par la qualité de la synthèse de nouvelles vues. Cet article propose d'utiliser la génération de nouvelles vues à partir d'un modèle 3DGS [3] pour affiner la calibration par rétropropagation de la loss de couleur. L'affinage de la calibration à lui seul apporte une amélioration moyenne de 0,4 dB PSNR sur le jeu de données utilisé comme référence par 3DGS [3]. Ce processus d'affinage peut être jugé trop long en fonction de la criticité du temps d'entraînement, mais pour la calibration de scènes de référence, telles que Mip-nerf 360 [1], l'enjeu de la qualité des nouvelles vues est primordial.

Mots Clef

Synthèse de vue, calibration, apprentissage.

Abstract

The quality of the camera calibration is of major importance for evaluating progresses in novel view synthesis, as a 1-pixel error on the calibration has a significant impact on the reconstruction quality. While there is no ground truth for real scenes, the quality of the calibration is assessed by the quality of the novel view synthesis. This paper proposes to use a 3DGS [3] model to fine tune calibration by backpropagation of novel view color loss with respect to the cameras parameters. The new calibration alone brings an average improvement of 0.4 dB PSNR on the dataset used as reference by 3DGS [3]. The fine tuning may be long and its suitability depends on the criticality of training time, but for calibration of reference scenes, such as [1], the stake of novel view quality is the most important.

Keywords

Example, model, template.

1 Introduction

3d gaussian splatting [3] has been a major breakthrough in the domain of 3D novel view synthesis, reaching state-

of-the-art quality with an efficient and realtime rendering performance. The research community has been flourishing since, offering continuous but modest improvements. For instance, MCMC [4] improvements range from 0.4 dB and 0.7 dB of PSNR (Peak Signal to Noise Ratio, a reference quality metric in novel view synthesis).

In such a context, the quality of the camera calibration is of major importance, as a 1-pixel error on the calibration has a significant impact on the reconstruction quality, sometimes up to 1 dB PSNR. The calibration is a field of research in itself, which does not focus on quality but seeks the best balance between precision, robustness and computational efficiency. The state of the art is held by feature based algorithms, with COLMAP [8] as a reference, and many proposals to improve the features tracks used in that algorithm (see e.g. [7]). Such solutions give a calibration suitable for novel view synthesis algorithms to converge. However, the calibration might introduce errors in some circumstances, e.g. low-textured scenes, or with uneven features distribution. The hard point about evaluating the quality of calibration is that there is no ground truth for real scenes. As such, the quality of the calibration is assessed by the quality of the novel view synthesis. Some publications leverage amortized learned neural networks, although mainly to improve processing performance. DUST3R [11] for example claims to avoid the cumbersome estimate of camera parameters, but does not claim better final quality.

Taking this into consideration, previous works propose to train the pose of the cameras based on the loss of the novel view synthesis itself, like gsplatloc [13] which generates a depthmap from a 3DGS model and compares it with a ground truth, or gsplat [12] which backpropagates on the pose of the camera during the 3DGS training. Similar to [12], we propose to train a 3DGS model from a pre-existing calibration, and fine-tune this calibration based on the color loss of the novel view synthesis. Unlike gsplat, we focus some iterations of the training on calibration improvement, replace L1 loss by a L2 loss in these iterations which performs better than L1, and train the focal length of the cameras alongside the pose. We also propose a reparameterization of the camera extrinsics and intrinsics for a more efficient training. These changes improve the final quality of the calibration, reaching an average 0.4 dB PSNR improvement on the reference dataset of 3DGS [3].

2 Method

This paper proposes to use backpropagation to train camera parameters from the color loss. Section 2.1 explains how backpropagation is simplified for implementation ease. The choice of the color loss is discussed in section 2.2. Then the training schedule between model and camera fine-tuning is the subject of 2.3. Finally, section 2.4 proposes a reparameterization of the camera model for a more performant training with Adam optimizer.

2.1 Backpropagation

The whole rendering is kept identical to 3DGS [3], except that the camera parameters are trained based on the color loss. We note the camera parameters $x_c, y_c, z_c, q, \phi_x, \phi_y$ where x_c, y_c, z_c and q are respectively the translation and rotation (as a quaternion) of the world-to-camera pose, and ϕ_x, ϕ_y are the fields of view along u, v axis. Noting p_j one parameter of camera j listed above, the derivatives of the color loss $\mathcal{L}$ with respect to p are:

$$\frac{\partial \mathcal{L}}{\partial p_j} = \sum_i \frac{\partial \mathcal{L}}{\partial uv_i^j} \frac{\partial uv_i^j}{\partial p_j} + \frac{\partial \mathcal{L}}{\partial cov2d_i^j} \frac{\partial cov2d_i^j}{\partial p_j} \quad (1)$$

With uv_i^j and $cov2d_i^j$ the projections on image plan of the camera j of respectively the center and the covariance matrix of the gaussian i .

We neglect the impact of p_j on $cov2d_i^j$ in (1), leading to:

$$\frac{\partial \mathcal{L}}{\partial p_j} = \sum_i \frac{\partial \mathcal{L}}{\partial uv_i^j} \frac{\partial uv_i^j}{\partial p_j} \quad (2)$$

This approximation is valid if the covariance of gaussian i is small compared to its distance to the camera j , which is the case for most gaussians in the model. Note that this approximation is already done by [3] when they approximate pinhole projection of the gaussian by an orthogonal projection. In practice, Colmap's lack of precision usually happens on the z_c position versus the ϕ_x, ϕ_y . Changing these parameters will have no impact on the transformation of the covariance from absolute referential to the relative referential, and small impact on the projection to the image plan.

2.2 Choice of the loss

While the choice of L1 loss (MAE: Mean Absolute Error) achieves competitive results in training gaussian model, we found empirically that L2 error (MSE: Mean Square Error) is more suited to train the camera parameters. The choice of the appropriate loss is a trade-off between the convergence performance of L2 and the robustness to outliers offered by L1. On a total of a few tens to few hundreds of cameras, a mis-calibrated camera may not be negligible for the training of the gaussians it sees. This explains the choice of L1 for 3DGS [3]. On the contrary, fine-tuning the calibration of one camera based on the millions of gaussians that compose the 3D model is less subject to outliers,

thus giving an advantage to the L2 loss. In practice, our experiments show that L1 can lead the system to diverge, some scenes ending up with trained cameras parameters worse than the input ones.

Using different losses for model training and camera calibration implies to dedicate some iterations to camera parameters training. The results are more precise and reliable, at the cost of an extended training duration. Though processing performance is not our main focus, we propose a training schedule in the next section to mitigate this problem.

2.3 Training schedule

The 3DGS model training uses the cameras calibration as an input, and vice versa. Concurrent training of inter-dependent parameters are common in machine learning, and gsplat [12] trains both the camera parameters and the 3DGS model in parallel. We propose a different schedule, by phase, with an early stopping mechanism to save compute time on cameras which do not benefit from fine-tuning. Specifically, we proceed in a loop with M iterations of 3DGS model training, followed by the training of all the cameras parameters, one camera after another. One camera is trained for at least 100 steps and maximum 1000 steps, a progress score being evaluated at every step and the training of the camera is interrupted when the progress score reaches a minimum threshold t .

The progress score s is an exponential moving average (EMA) of the PSNR progress:

$$s_{n+1} = s_n(1 - \beta) + \beta(PSNR_{n+1} - PSNR_n) \quad (3)$$

In our experiments, we initialize the score with $s_0 = 0$ and use $\beta = \frac{1}{50}$. This ensures that after the first 100 steps the EMA is a good approximation of the average progress. Letting $M = 3000$ model training steps between cameras fine-tuning is the natural pace of 3DGS [3] training, as it is the default interval between opacity reset. We experimentally found that the threshold $t = 0.0002$ was a good trade-off between training duration and final quality.

This schedule enables us to save training iterations on cameras whose training does not improve the PSNR. In our experiments, this leads to 20% savings in cameras training time.

2.4 Reparameterization

Consider the pinhole projection of the center of one gaussian whose position relative to the camera is x, y, z :

$$(u, v) = \left(\frac{f_x}{z} x, \frac{f_y}{z} y \right) \quad (4)$$

With f_x, f_y the focal lengths of the camera, along u, v axis. We can see from eq. 4 that any modification of the camera parameters that keeps $(f_x/z, f_y/z)$ constant will have no impact of (u, v) . If all gaussians had the same z , it would not be possible to disentangle the parameters. Though this case never arises, f_x, f_y and z are still highly correlated.

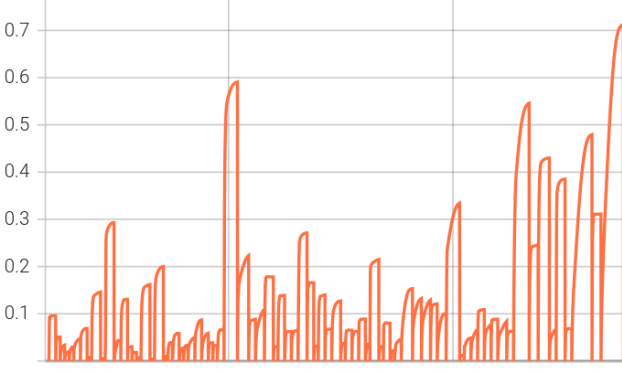


Figure 1: Tracking of the PSNR improvement during the first phase of cameras fine-tuning. Each increasing sub-graph concerns one camera. One can see that the duration of training depends on the improvement pace. Y axis is the PSNR improvement of the current camera, x axis is the iteration.

This correlation is related to the similarity between getting closer to the scene and zooming in with the camera. Note that in 3DGS [3], the focal f_x, f_y parameters are replaced by the field of view ϕ_x, ϕ_y .

To help training the model, the parameters (z_c, ϕ_x, ϕ_y) of a camera are replaced by (a, b, c) . One reparameterization is to choose (a, b, c) as the coordinates in the referential defined by the eigen vectors of the Hessian matrix $\mathcal{H}$ of $\mathcal{L}(z_c, \phi_x, \phi_y)$. Calling $\mathcal{E}$ this matrix, we have:

$$\begin{bmatrix} z_c \\ \phi_x \\ \phi_y \end{bmatrix} = \begin{bmatrix} z_c^0 \\ \phi_x^0 \\ \phi_y^0 \end{bmatrix} + [\mathcal{E}] \begin{bmatrix} a \\ b \\ c \end{bmatrix} \quad (5)$$

The rationale behind this reparameterization is that in the neighborhood of the local minimum, the gradient of the loss is approximated by $\frac{\partial \mathcal{L}}{\partial \theta} \approx \mathcal{H}\theta$. By definition, in referential given by the Hessian eigen vectors, the Hessian matrix is diagonal, thus the derivative with respect to one parameter is independent from the other parameters: $\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \lambda_i \theta_i$ (with λ_i the i -th eigen value). The Adam optimizer [5] uses momentum, which is a way to accumulate a velocity vector in directions of persistent loss reduction [9]. In the directions of low curvature, the loss gradient tends to persist across iterations and be amplified by momentum accumulation. This enables efficient convergence for parameters of low impact (small eigen values of the Hessian), that would otherwise be highly disturbed by high impact parameters.

In practice, the first optim phase is done with the standard (z_c, ϕ_x, ϕ_y) parameters, then the values of $(z_c^0, \phi_x^0, \phi_y^0)$ are set, the hessian is evaluated, (a, b, c) are initiated at $(0, 0, 0)$ and the rest of the optimization process is done with (x_c, y_c, q, a, b, c) parameters.

Since the second derivation of the loss gives unstable results, we approximate the hessian by finite difference, evaluated from the gradient on the current (z_c, ϕ_x, ϕ_y) and on points around it $(z_c \pm \epsilon, \phi_x \pm \epsilon, \phi_y \pm \epsilon)$.

3 Results

Our experiments are processed in two steps: we first train the camera positions and model together, as described in 2, then we re-train the model, render the test images and process metrics with the original code of [3]. The results are listed in Table 1. Table 2 gives the number of gaussians of the 3DGS [3] for the same scenes. The fine-tuned calibration enables generally a better PSNR with less gaussians. The few images with worse PSNR are points of views shared by so few cameras that the reconstruction is not reliable. Cf Figure 5.

scene	dataset calibration PSNR [dB]	ours PSNR [dB]
mip360 bicycle	25.61	26.34
mip360 bonsai	32.32	32.72
mip360 counter	29.11	29.26
mip360 garden	27.77	28.32
mip360 kitchen	31.55	31.50
mip360 room	31.72	31.94
mip360 stump	26.90	27.28
T&T Train	22.12	22.61
T&T Truck	25.84	26.18
DB DrJohnson	29.11	30.12
DB Playroom	29.96	30.77
Average	28.36	28.79

Table 1: PSNR on the reference scenes as defined by 3DGS [3]. Column "dataset calibration" lists results of training using the Colmap calibration provided with the datasets. Column "ours" lists results of training with the exact same code but our calibration.

scene	dataset calibration gaussians [M]	ours gaussians [M]
mip360 bicycle	5.79	4.72
mip360 bonsai	1.25	1.14
mip360 counter	1.17	1.13
mip360 garden	5.09	3.63
mip360 kitchen	1.74	1.54
mip360 room	1.49	1.38
mip360 stump	4.66	4.26
T&T Train	1.01	1.13
T&T Truck	2.31	2.03
DB DrJohnson	3.31	3.34
DB Playroom	2.31	1.96
Average	2.35	2.06

Table 2: Number of gaussians (in millions) on the reference scenes as defined by 3DGS [3]. Column "dataset calibration" lists results of training using the Colmap calibration provided with the datasets. Column "ours" lists results of training with the exact same code but our calibration.

On some images, one can see the slight calibration error of the Colmap input, and the result of the fine-tuning. Figure 2 shows a slight misalignment on Colmap [8], visible on

the wheel of the bicycle [1], which is corrected by camera fine-tuning. Figure 3 shows another example on DrJohnson [2] where we have used RAFT [10] motion flow to help visualize the improvement between the calibration given with the datasets (ref) and the fine-tuned calibration (ours). Figure 4 displays the motion histogram, in pixels.

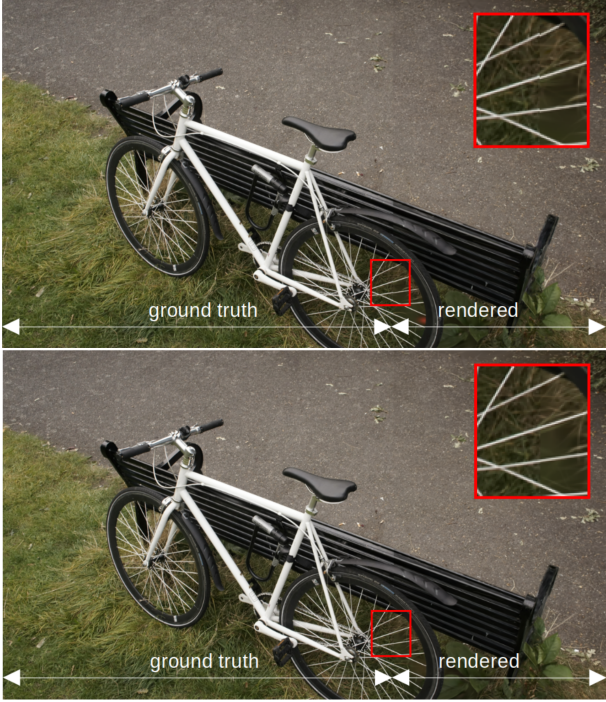


Figure 2: Reference (top) vs ours (bottom) on bicycle view.

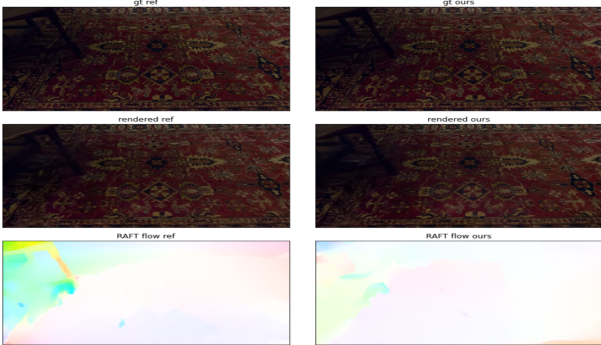


Figure 3: Reference (left) vs ours (right) on DrJohnson view. Bottom: motion flow estimated with RAFT [10]

3.1 Ablations

Table 3 gives the PSNR obtained by the same process as ours, but with one of the aspects missing.

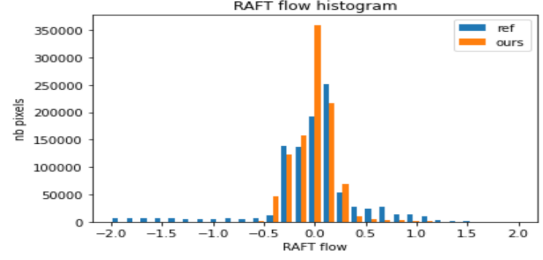


Figure 4: Reference (blue) vs ours (orange) motion flow histogram on DrJohnson view. Motion is in pixels.

- w/o abc does not use reparameterization.
- w/o fov does not train (ϕ_x, ϕ_y) . Note that this implies not to use reparameterization neither.
- L1 uses L1 loss instead of L2, for camera parameters training.
- once runs only on phase of cameras fine-tuning, instead of one phase every 3000 3DGS steps

scene	w/o abc	w/o fov	L1	once	full
bicycle	25.81	25.74	25.82	25.62	26.34
bonsai	32.31	32.15	32.41	32.32	32.72
counter	29.04	29.09	29.13	29.25	29.26
garden	27.70	27.50	27.72	27.87	28.32
kitchen	31.12	31.07	30.71	31.61	31.14
room	31.75	31.77	31.80	32.03	31.94
stump	26.72	26.60	26.73	26.88	27.28
Train	22.47	22.31	22.54	22.46	22.61
Truck	26.13	26.05	26.11	25.86	26.18
DrJohnson	29.93	29.55	29.81	29.47	30.12
Playroom	30.61	30.36	30.45	29.99	30.77
Average	28.51	28.38	28.48	28.49	28.79

Table 3: Ablation: PSNR [dB] with some of the solution missing. Scenes bicycle, bonsai, counter, garden, kitchen, room and stump from Mip-nerf 360 [1], Train and Truck from Tanks and temples [6], DrJohnson and Playroom from Deep Blending [2].

4 Conclusion

Considering the precision of camera calibration as a matter of importance for the comparison of publications of novel view synthesis, we propose a solution to fine tune this calibration. The camera calibration uses a 3DGS model which is trained in parallel in an interleaved training schedule. The new calibration alone brings an average improvement of 0.4 dB PSNR on the dataset used as reference by 3DGS [3]. The fine tuning may be long and its suitability depends on the criticality of training time, but for calibration of reference scenes, such as Mip-nerf 360 [1], the stake of novel view quality is the most important.



Figure 5: Example of failure. A detailed zone (the tree), seen by few cameras lead to bad reconstruction, which disturbs camera fine-tuning.

References

- [1] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5470–5479, 2022.
- [2] Peter Hedman, Julien Philip, True Price, Jan-Michael Frahm, George Drettakis, and Gabriel Brostow. Deep blending for free-viewpoint image-based rendering. *ACM Transactions on Graphics (SIGGRAPH Asia Conference Proceedings)*, 37(6), November 2018.
- [3] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.
- [4] Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Jeff Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *arXiv preprint arXiv:2404.09591*, 2024.
- [5] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [6] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics (ToG)*, 36(4):1–13, 2017.
- [7] Philipp Lindenberger, Paul-Edouard Sarlin, Viktor Larsson, and Marc Pollefeys. Pixel-perfect structure-from-motion with featuremetric refinement. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5987–5997, 2021.
- [8] Johannes L Schönberger. *Robust methods for accurate and efficient 3D modeling from unstructured imagery*. PhD thesis, ETH Zurich, 2018.
- [9] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International conference on machine learning*, pages 1139–1147. pmlr, 2013.
- [10] Zachary Teed and Jia Deng. RAFT: recurrent all-pairs field transforms for optical flow. *CoRR*, abs/2003.12039, 2020.
- [11] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20697–20709, 2024.
- [12] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, et al. gsplat: An open-source library for gaussian splatting. *arXiv preprint arXiv:2409.06765*, 2024.
- [13] Atticus J Zeller. Gsplatloc: Ultra-precise camera localization via 3d gaussian splatting. *arXiv preprint arXiv:2412.20056*, 2024.